

Gaussian Integral Linear Operators for Precomputed Graphics: Supplementary Material

HAOLIN LU, Max Planck Institute for Informatics, Germany

YASH BELHE, University of California San Diego, USA

GURPRIT SINGH, Max Planck Institute for Informatics, Germany

TZU-MAO LI, University of California San Diego, USA

TOSHIYA HACHISUKA, University of Waterloo, Canada

ACM Reference Format:

Haolin Lu, Yash Belhe, Gurprit Singh, Tzu-Mao Li, and Toshiya Hachisuka. 2025. Gaussian Integral Linear Operators for Precomputed Graphics: Supplementary Material. *ACM Trans. Graph.* 44, 6, Article 251 (December 2025), 11 pages. <https://doi.org/10.1145/3763321>

1 OVERVIEW

In [Section 2](#), we review the fundamental properties of Gaussians to support further derivation. [Section 3](#) then derive how Gaussian integral linear operator can be solved and composed in closed form, and how to further supporting the diagonal variation. Then, [Section 4](#), [Section 5](#), and [Section 6.1](#) discuss the details in precomputed Poisson kernel, precomputed radiance transfer (PRT), and spectral rendering applications respectively.

2 FUNDAMENTAL PROPERTIES OF GAUSSIANS

Definition. An N -dimensional Gaussian distribution is characterized by its mean μ and covariance matrix Σ :

$$\mathcal{N}(\mu, \Sigma) = \frac{1}{(2\pi)^{n/2} \cdot \sqrt{|\Sigma|}} \exp \left[-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right]. \quad (1)$$

In the following derivation, we sometimes partition the N -dimensional space into two subsets: $\mathbf{x} \in \mathbb{R}^{N_x}$ and $\mathbf{y} \in \mathbb{R}^{N_y}$, such that $N = N_x + N_y$. We consider a function $f(\mathbf{x}, \mathbf{y})$ defined over this joint space, which corresponds to the probability density function (PDF) of an N -dimensional Gaussian distribution:

$$f \left(\begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix} \right) = \mathcal{N} \left(\begin{bmatrix} \mu_x \\ \mu_y \end{bmatrix}, \begin{bmatrix} \Sigma_{xx} & \Sigma_{xy} \\ \Sigma_{xy}^\top & \Sigma_{yy} \end{bmatrix} \right). \quad (2)$$

Authors' addresses: Haolin Lu, Max Planck Institute for Informatics, Germany; Yash Belhe, vicini@google.com, University of California San Diego, USA; Gurprit Singh, Max Planck Institute for Informatics, Germany; Tzu-Mao Li, University of California San Diego, USA; Toshiya Hachisuka, University of Waterloo, Canada.

Please use nonacm option or ACM Engage class to enable CC licenses



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 0730-0301/2025/12-ART251

<https://doi.org/10.1145/3763321>

Marginalization. Integrating the function f (Eq. (2)) over $\mathbf{x}$ or $\mathbf{y}$ corresponds to marginalization in a Gaussian distribution. As a result, the integral yields another normalized Gaussian distribution of lower dimensionality:

$$f_x(x) = \int f(\mathbf{x}, \mathbf{y}) d\mathbf{y} = \mathcal{N}(x; \mu_x, \Sigma_{xx}) \quad (3)$$

$$f_y(y) = \int f(\mathbf{x}, \mathbf{y}) d\mathbf{x} = \mathcal{N}(y; \mu_y, \Sigma_{yy}). \quad (4)$$

Conditioning. Fixing $\mathbf{x}$ or $\mathbf{y}$ to constant values, $\mathbf{x}_1$ or $\mathbf{y}_1$, corresponds to conditioning in a Gaussian distribution, which also results in a lower-dimensional unnormalized Gaussian distribution:

$$f_{\mathbf{x}|\mathbf{y}=\mathbf{y}_1}(\mathbf{x}) = f(\mathbf{x}, \mathbf{y} = \mathbf{y}_1) = C_{\mathbf{x}|\mathbf{y}} \mathcal{N}(\mathbf{x}; \mu_{\mathbf{x}|\mathbf{y}}, \Sigma_{\mathbf{x}|\mathbf{y}}) \quad (5)$$

$$f_{\mathbf{y}|\mathbf{x}=\mathbf{x}_1}(\mathbf{y}) = f(\mathbf{y}, \mathbf{x} = \mathbf{x}_1) = C_{\mathbf{y}|\mathbf{x}} \mathcal{N}(\mathbf{y}; \mu_{\mathbf{y}|\mathbf{x}}, \Sigma_{\mathbf{y}|\mathbf{x}}), \quad (6)$$

where the new parameters can be computed as:

$$C_{\mathbf{x}|\mathbf{y}} = f_{\mathbf{y}}(\mathbf{y} = \mathbf{y}_1) = \mathcal{N}(\mathbf{y} = \mathbf{y}_1; \mu_y, \Sigma_{yy}) \quad (7)$$

$$C_{\mathbf{y}|\mathbf{x}} = f_{\mathbf{x}}(\mathbf{x} = \mathbf{x}_1) = \mathcal{N}(\mathbf{x} = \mathbf{x}_1; \mu_x, \Sigma_{xx}) \quad (8)$$

$$\mu_{\mathbf{x}|\mathbf{y}} = \mu_x + \Sigma_{xy} \Sigma_{yy}^{-1} (\mathbf{y} - \mu_y) \quad (9)$$

$$\mu_{\mathbf{y}|\mathbf{x}} = \mu_y + \Sigma_{xy}^T \Sigma_{xx}^{-1} (\mathbf{x} - \mu_x) \quad (10)$$

$$\Sigma_{\mathbf{x}|\mathbf{y}} = \Sigma_{xx} - \Sigma_{xy} \Sigma_{yy}^{-1} \Sigma_{xy}^T \quad (11)$$

$$\Sigma_{\mathbf{y}|\mathbf{x}} = \Sigma_{yy} - \Sigma_{xy}^T \Sigma_{xx}^{-1} \Sigma_{xy} \quad (12)$$

Multiplication. The product of two N-dim Gaussians is also known to be an unnormalized Gaussian:

$$\mathcal{N}(\mu_a, \Sigma_a) \cdot \mathcal{N}(\mu_b, \Sigma_b) = z_{a \cdot b} \mathcal{N}(\mu_{a \cdot b}, \Sigma_{a \cdot b}), \quad (13)$$

where the new parameters of the result Gaussian is defined as:

$$\mu_{a \cdot b} = \Sigma_{a \cdot b} \cdot (\Sigma_a^{-1} \mu_a + \Sigma_b^{-1} \mu_b) \quad (14)$$

$$\Sigma_{a \cdot b} = (\Sigma_a^{-1} + \Sigma_b^{-1})^{-1} \quad (15)$$

$$z_{a \cdot b} = (2\pi)^{-D/2} |\Sigma_a + \Sigma_b|^{-1/2} \cdot \exp\left(-\frac{1}{2} (\mu_a - \mu_b)^T (\Sigma_a + \Sigma_b)^{-1} (\mu_a - \mu_b)\right) \quad (16)$$

3 PROOFS OF GAUSSIAN LINEAR OPERATOR

We now examine the key mathematical properties underlying our method. The most fundamental is the analytical solvability of Gaussian-based integral linear operators. Specifically, it says the product of a Gaussian in $(N_x + N_y)$ dimensions and another in N_y dimensions can be analytically integrated over $\mathbf{y}$, resulting in a new Gaussian in $\mathbf{x}$:

$$\boxed{z \cdot \mathcal{N}(\mathbf{x}) = \int \mathcal{N}(\mathbf{x}, \mathbf{y}) \cdot \mathcal{N}(\mathbf{y}) d\mathbf{y}.} \quad (17)$$

We also rely on the fact that two Gaussian integral linear operators can be composed analytically. Composition linear operators involves integrating the product of two kernels over their shared variable:

$$K_{12}(\mathbf{x}, \mathbf{z}) = \int K_1(\mathbf{x}, \mathbf{y}), K_2(\mathbf{y}, \mathbf{z}), d\mathbf{y}. \quad (18)$$

In the Gaussian case, this means that if both K_1 and K_2 are Gaussians, their composition yields another Gaussian:

$$z \cdot \mathcal{N}(\mathbf{x}, z) = \int \mathcal{N}(\mathbf{x}, y) \cdot \mathcal{N}(y, z) dy. \quad (19)$$

Lastly, in our spectral rendering example, we need to handle diagonal components with a Dirac delta term, $\delta(y = z)$ (or $\delta(y - z)$), which signifies that values exist only along the diagonal. We first consider the composition of a non-diagonal operator with a diagonal operator, which we will later show results in another Gaussian:

$$z \cdot \mathcal{N}(\mathbf{x}, z) = \int \mathcal{N}(\mathbf{x}, y) \cdot \mathcal{N}(z) \cdot \delta(y - z) dy = \mathcal{N}(\mathbf{x}, z) \cdot \mathcal{N}(z). \quad (20)$$

Next, we consider the composition of two diagonal operators analytically:

$$z \cdot \mathcal{N}(\mathbf{x}) \cdot \delta(\mathbf{x} - z) = \int \mathcal{N}_1(\mathbf{x}) \cdot \delta(\mathbf{x} - y) \cdot \mathcal{N}_2(z) \cdot \delta(y - z) dy = \mathcal{N}_1(\mathbf{x}) \cdot \mathcal{N}_2(\mathbf{x}) \cdot \delta(\mathbf{x} - z), \quad (21)$$

which simplifies to the known result that the product of two equal-dimensional Gaussians is another Gaussian, as shown in Eq. (13).

Finally, we consider applying a diagonal operator to an input Gaussian analytically:

$$z \cdot \mathcal{N}(\mathbf{x}) = \int \mathcal{N}_1(\mathbf{x}) \cdot \delta(\mathbf{x} - y) \cdot \mathcal{N}_2(y) dy = \mathcal{N}_1(\mathbf{x}) \cdot \mathcal{N}_2(\mathbf{x}), \quad (22)$$

which again reduces to the standard Gaussian product form given in Eq. (13).

Therefore, only Eq. (17), Eq. (19), and Eq. (20) require new proofs. Below, we start with Eq. (19) first:

Lemma 1. Given two functions, $A(\mathbf{x}, y)$ and $b(y)$, both correspond to the PDF of a normalized Gaussian:

$$A\left(\begin{bmatrix} \mathbf{x} \\ y \end{bmatrix}\right) = \mathcal{N}\left(\begin{bmatrix} \mu_x \\ \mu_y \end{bmatrix}, \begin{bmatrix} \Sigma_{xx} & \Sigma_{xy} \\ \Sigma_{xy}^\top & \Sigma_{yy} \end{bmatrix}\right), \quad (23)$$

$$b(y) = \mathcal{N}(\mu^b, \Sigma^b), \quad (24)$$

then their product will be another unnormalized Gaussian:

$$f(\mathbf{x}, y) = A(\mathbf{x}, y) \cdot b(y) = C \cdot \mathcal{N}(\mu, \Sigma). \quad (25)$$

PROOF. For simplicity, let's denote the inverse of the block matrix as below:

$$\begin{bmatrix} \Sigma_{xx}^{A-1} & \Sigma_{xy}^{A-1} \\ \Sigma_{xy}^{A-1\top} & \Sigma_{yy}^{A-1} \end{bmatrix} = \begin{bmatrix} \Sigma_{xx}^A & \Sigma_{xy}^A \\ \Sigma_{xy}^{A\top} & \Sigma_{yy}^A \end{bmatrix}^{-1}, \quad (26)$$

it is a bit abuse of notation, and please notice in general $\Sigma_{xx}^{A-1} \neq (\Sigma_{xx}^A)^{-1}$ to avoid potential confusion.

We first focus on the term inside the exponential and ignore the constant -0.5 . We first assume that this exponent of $A(\mathbf{x}, y) \cdot b(y)$ can be combined into one single quadratic form, that is:

$$\begin{bmatrix} \mathbf{x} - \mu_x \\ y - \mu_y \end{bmatrix}^T \begin{bmatrix} \Sigma_{xx}^{A-1} & \Sigma_{xy}^{A-1} \\ \Sigma_{xy}^{A-1\top} & \Sigma_{yy}^{A-1} \end{bmatrix} \begin{bmatrix} \mathbf{x} - \mu_x \\ y - \mu_y \end{bmatrix} + (y - \mu^b)^T \Sigma^{b-1} (y - \mu^b) \quad (27)$$

$$= \begin{bmatrix} \mathbf{x} - \mathbf{a} \\ y - \mathbf{b} \end{bmatrix}^T \begin{bmatrix} \Sigma^{11} & \Sigma^{12} \\ \Sigma^{21} & \Sigma^{22} \end{bmatrix} \begin{bmatrix} \mathbf{x} - \mathbf{a} \\ y - \mathbf{b} \end{bmatrix} + c \quad (28)$$

To start with, we consider the values of Σ^{11} , Σ^{12} , Σ^{21} and Σ^{22} . Actually, We can directly infer their values by comparing the quadratic terms. For instance, consider the term involving $\mathbf{x}^T \Sigma \mathbf{x}$. In Eq. (27), this term is $\mathbf{x}^T \Sigma_{xx}^{A-1} \mathbf{x}$,

while in Eq. (28), it is $\mathbf{x}^T \Sigma^{11} \mathbf{x}$. Thus, for Eq. (28) to hold, Σ^{11} must equal Σ_{xx}^{A-1} . For the same reason, we can easily find that, if Eq. (27) can be written as Eq. (28), the Σ^{ij} must be:

$$\begin{bmatrix} \Sigma^{11} & \Sigma^{12} \\ \Sigma^{21} & \Sigma^{22} \end{bmatrix} = \begin{bmatrix} \Sigma_{xx}^{A-1} & \Sigma_{xy}^{A-1} \\ \Sigma_{yx}^{A-1} & \Sigma_{yy}^{A-1} + \Sigma^{b-1} \end{bmatrix} \quad (29)$$

Next, we determine the parameters $\mathbf{a}$, $\mathbf{b}$, by solving a linear system derived from equating the linear terms in $(\dots)\mathbf{x}$ and $(\dots)\mathbf{y}$ of Eq. (27) and Eq. (28):

$$\Sigma_{xx}^{A-1} \mu_x + \Sigma_{xy}^{A-1} \mu_y = \Sigma_{xx}^{A-1} \mathbf{a} + \Sigma_{xy}^{A-1} \mathbf{b} \quad (30)$$

$$\Sigma^{b-1} \mu^b + \Sigma_{yy}^{A-1} \mu_y + \Sigma_{yx}^{A-1} \mu_x = \Sigma_{yx}^{A-1} \mathbf{a} + (\Sigma_{yy}^{A-1} + \Sigma^{b-1}) \mathbf{b} \quad (31)$$

Notice that for PDF of Gaussian to exist, its covariance matrix Σ must be positive definite, and its inverse Σ^{-1} is thus also guaranteed to be positive definite. As the principal submatrices of $(\Sigma^A)^{-1}$, Σ_{xx}^{A-1} is thus known to be positive definite. Therefore, we can left multiply Eq. (30) by $\Sigma_{yx}^{A-1} (\Sigma_{xx}^{A-1})^{-1}$ to obtain.

$$\Sigma_{yx}^{A-1} (\Sigma_{xx}^{A-1})^{-1} (\Sigma_{xx}^{A-1} \mu_x + \Sigma_{xy}^{A-1} \mu_y) = \Sigma_{yx}^{A-1} \mathbf{a} + \Sigma_{yx}^{A-1} (\Sigma_{xx}^{A-1})^{-1} \Sigma_{xy}^{A-1} \mathbf{b} \quad (32)$$

By subtracting Eq. (32) from Eq. (31), we can obtain an expression only about $\mathbf{b}$:

$$\Sigma^{b-1} \mu^b + \Sigma_{yy}^{A-1} \mu_y - \Sigma_{yx}^{A-1} (\Sigma_{xx}^{A-1})^{-1} \Sigma_{xy}^{A-1} \mu_y = (\Sigma_{yy}^{A-1} + \Sigma^{b-1} - \Sigma_{yx}^{A-1} (\Sigma_{xx}^{A-1})^{-1} \Sigma_{xy}^{A-1}) \mathbf{b}, \quad (33)$$

We now show that, the matrix of the right hand side, $(\Sigma_{yy}^{A-1} + \Sigma^{b-1} - \Sigma_{yx}^{A-1} (\Sigma_{xx}^{A-1})^{-1} \Sigma_{xy}^{A-1})$ is actually invertible. The core observation is that $(\Sigma_{yy}^{A-1} - \Sigma_{yx}^{A-1} (\Sigma_{xx}^{A-1})^{-1} \Sigma_{xy}^{A-1})$ is the *schur complement* of a symmetric positive definite matrix, and therefore it is also positive definite. Since Σ^{b-1} is positive definite itself, the entire term is just a sum of two positive definite matrices, which is naturally also positive definite and thus invertible. As a result, we can directly solve $\mathbf{a}$ directly by invert the matrix and obtain:

$$\mathbf{b} = [\Sigma^{b-1} + \Sigma_{\psi}]^{-1} [\Sigma^{b-1} \mu^b + \Sigma_{\psi} \mu_y], \quad (34)$$

$$\text{where } \Sigma_{\psi} = \Sigma_{yy}^{A-1} - \Sigma_{yx}^{A-1} (\Sigma_{xx}^{A-1})^{-1} \Sigma_{xy}^{A-1} \quad (35)$$

By substitute $\mathbf{b}$ into Eq. (30), $\mathbf{a}$ can also be solved by invert Σ_{xx}^{A-1} :

$$\mathbf{a} = \mu_x + (\Sigma_{xx}^{A-1})^{-1} \Sigma_{xy}^{A-1} (\mu_y - \mathbf{b}). \quad (36)$$

Upon this point, all linear and quadratic terms are perfectly matched, so we can concludes that Eq. (27) can indeed be written as Eq. (28). The only missing term is the constant c , which has:

$$\mathbf{b}^T (\Sigma_{yy}^{A-1} + \Sigma^{b-1}) \mathbf{b} + 2\mathbf{b}^T \Sigma_{yx}^{A-1} \mathbf{a} + \mathbf{a}^T \Sigma_{xx}^{A-1} \mathbf{a} + c = \mu_y^T \Sigma_{yy}^{A-1} \mu_y + 2\mu_y^T \Sigma_{yx}^{A-1} \mu_x + \mu_x^T \Sigma_{xx}^{A-1} \mu_x + \mu^b{}^T \Sigma^{b-1} \mu^b \quad (37)$$

$$c = (\mu_y^T \Sigma_{yy}^{A-1} \mu_y + 2\mu_y^T \Sigma_{yx}^{A-1} \mu_x + \mu_x^T \Sigma_{xx}^{A-1} \mu_x + \mu^b{}^T \Sigma^{b-1} \mu^b) - (\mathbf{b}^T (\Sigma_{yy}^{A-1} + \Sigma^{b-1}) \mathbf{b} + 2\mathbf{b}^T \Sigma_{yx}^{A-1} \mathbf{a} + \mathbf{a}^T \Sigma_{xx}^{A-1} \mathbf{a}) \quad (38)$$

Then, we consider the entire term, by computing the extra constant C . By putting c into $\exp(-\frac{c}{2})$, and by listing the normalizing term of Eq. (27) and Eq. (28), we can obtain:

$$\frac{1}{(2\pi)^{(N_x+N_y)/2} \sqrt{|\Sigma^A|}} \cdot \frac{1}{(2\pi)^{N_y/2} \sqrt{|\Sigma^b|}} = C \cdot \frac{1}{(2\pi)^{(N_x+N_y)/2} \sqrt{|\Sigma|}} / \exp(-\frac{c}{2}) \quad (39)$$

$$C = \frac{\exp(-c/2) \cdot \sqrt{|\Sigma|}}{(2\pi)^{N_y/2} \sqrt{|\Sigma^A|} \cdot \sqrt{|\Sigma^b|}}. \quad (40)$$

To conclude:

$$C \cdot \mathcal{N}\left(\begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix} \middle| \mu, \Sigma\right) = \mathcal{N}\left(\begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix} \middle| \begin{bmatrix} \mu_x \\ \mu_y \end{bmatrix}, \begin{bmatrix} \Sigma_{xx} & \Sigma_{xy} \\ \Sigma_{xy}^T & \Sigma_{yy} \end{bmatrix}\right) \cdot \mathcal{N}\left(\mathbf{y} \middle| \mu^b, \Sigma^b\right), \quad (41)$$

where $\mu = \begin{bmatrix} a \\ b \end{bmatrix}$, $\Sigma = \begin{bmatrix} \Sigma_{xx}^{A-1} & \Sigma_{xy}^{A-1} \\ \Sigma_{yx}^{A-1} & \Sigma_{yy}^{A-1} + \Sigma^{b-1} \end{bmatrix}^{-1}$, and C is given by Eq. (40). $\square$

Lemma 2. Given two functions, $A(\mathbf{x}, \mathbf{y})$ and $b(\mathbf{y})$, both correspond to the PDF of a normalized Gaussian:

$$A\left(\begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix}\right) = \mathcal{N}\left(\begin{bmatrix} \mu_x \\ \mu_y \end{bmatrix}, \begin{bmatrix} \Sigma_{xx} & \Sigma_{xy} \\ \Sigma_{xy}^\top & \Sigma_{yy} \end{bmatrix}\right), \quad (42)$$

$$b(\mathbf{y}) = \mathcal{N}(\mu^b, \Sigma^b), \quad (43)$$

then their integral will be another unnormalized Gaussian:

$$f(\mathbf{x}) = \int A(\mathbf{x}, \mathbf{y}) \cdot b(\mathbf{y}) \, d\mathbf{y} = C \cdot \mathcal{N}(\mu, \Sigma). \quad (44)$$

PROOF. Given what have been proven in Theorem 1, the integrand (product) is a Gaussian, so the integral is simply a marginalization of Gaussian, and it is easy to see: $\mu = \mathbf{a}$ defined in Eq. (36), Σ is the left-top submatrix of the inverse of what we obtained from Eq. (28), and C is similar to what defined in Eq. (40). $\square$

Lemma 3. Given two functions, $A(\mathbf{x}, \mathbf{y})$ and $B(\mathbf{x}, \mathbf{z})$, both correspond to the PDF of a normalized Gaussian:

$$A\left(\begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix}\right) = \mathcal{N}\left(\begin{bmatrix} \mu_x^A \\ \mu_y^A \end{bmatrix}, \begin{bmatrix} \Sigma_{xx}^A & \Sigma_{xy}^A \\ \Sigma_{xy}^{A^\top} & \Sigma_{yy}^A \end{bmatrix}\right), \quad (45)$$

$$B\left(\begin{bmatrix} \mathbf{x} \\ \mathbf{z} \end{bmatrix}\right) = \mathcal{N}\left(\begin{bmatrix} \mu_x^B \\ \mu_z^B \end{bmatrix}, \begin{bmatrix} \Sigma_{xx}^B & \Sigma_{xz}^B \\ \Sigma_{xz}^{B^\top} & \Sigma_{zz}^B \end{bmatrix}\right). \quad (46)$$

then their integral over $\mathbf{x}$ will be another unnormalized Gaussian:

$$f(\mathbf{y}, \mathbf{z}) = \int A(\mathbf{x}, \mathbf{y}) \cdot B(\mathbf{x}, \mathbf{z}) \, d\mathbf{x} = C \cdot \mathcal{N}(\mu, \Sigma). \quad (47)$$

PROOF. Given a pair of $[\mathbf{y}, \mathbf{z}]$ value, f is only an expression of $\mathbf{x}$, and can thus be written as product of conditional distributions. In derivation below, to highlight the connection with $[\mathbf{y}, \mathbf{z}]$, we use **blue** to denote expressions dependent on $\mathbf{y}$, **red** for those that depend on $\mathbf{z}$, and **purple** for those related to both:

$$f(\mathbf{y}, \mathbf{z}) = \int A(\mathbf{x}, \mathbf{y}) \cdot B(\mathbf{x}, \mathbf{z}) \, d\mathbf{x} = \int A(\mathbf{x}|\mathbf{y}) \cdot B(\mathbf{x}|\mathbf{z}) \, d\mathbf{x} \quad (48)$$

$$= \int \mathbf{c}_1 \cdot \mathcal{N}(\mathbf{x}|\mu_1, \Sigma_1) \times \mathbf{c}_2 \cdot \mathcal{N}(\mathbf{x}|\mu_2, \Sigma_2) \, d\mathbf{x}, \quad (49)$$

where the constants are defined from Eq. (7) to Eq. (12).

These two conditional Gaussians can be merged using Eq. (13):

$$f(\mathbf{y}, \mathbf{z}) = \int \mathbf{c}_1 \cdot \mathcal{N}(\mathbf{x}|\mu_1, \Sigma_1) \times \mathbf{c}_2 \cdot \mathcal{N}(\mathbf{x}|\mu_2, \Sigma_2) \, d\mathbf{x} \quad (50)$$

$$= \int \mathbf{c}_1 \cdot \mathbf{c}_2 \cdot \mathbf{c}_3 \cdot \mathcal{N}(\mathbf{x}|\mu_3, \Sigma_3) \, d\mathbf{x} \quad (51)$$

$$= \mathbf{c}_1 \cdot \mathbf{c}_2 \cdot \mathbf{c}_3. \quad (52)$$

We first consider $\mathbf{c}_1 \cdot \mathbf{c}_2$, according to Eq. (5),

$$\mathbf{c}_1 \cdot \mathbf{c}_2 = \mathcal{N}(\mathbf{y}; \mu_y^A, \Sigma_{yy}^A) \cdot \mathcal{N}(\mathbf{z}; \mu_z^B, \Sigma_{zz}^B) \quad (53)$$

Since $\mathbf{y}$ and $\mathbf{z}$ are independent dimensions, it can easily be written as a joint Gaussian distribution with diagonal covariance matrix:

$$\mathbf{c}_1 \mathbf{c}_2 = \mathcal{N} \left(\begin{bmatrix} \mathbf{y} \\ \mathbf{z} \end{bmatrix}; \begin{bmatrix} \mu_y^A \\ \mu_z^B \end{bmatrix}, \begin{bmatrix} \Sigma_{yy}^A & 0 \\ 0 & \Sigma_{zz}^B \end{bmatrix} \right). \quad (54)$$

Then we investigate $\mathbf{c}_3$, according to Eq. (13):

$$\mathbf{c}_3 = (2\pi)^{-D/2} |\Sigma_1 + \Sigma_2|^{-1/2} \quad (55)$$

$$\cdot \exp\left(-\frac{1}{2}(\mu_1 - \mu_2)^T (\Sigma_1 + \Sigma_2)^{-1} (\mu_1 - \mu_2)\right), \quad (56)$$

where D is the dimension of $\mathbf{x}$, and based on Eq. (5),

$$\mu_1 = \mu_x^A + \Sigma_{xy}^A \Sigma_{yy}^{A-} (\mathbf{y} - \mu_y^A) = M_4^A \mathbf{y} - \mu_4^A, \quad (57)$$

$$\mu_2 = \mu_x^B + \Sigma_{xz}^B \Sigma_{zz}^{B-} (\mathbf{z} - \mu_z^B) = M_4^B \mathbf{z} - \mu_4^B, \quad (58)$$

where we define M_4^A, μ_4^A and M_4^B, μ_4^B to simplify expressions.

For simplicity, we focus on the exponent part besides -0.5 , which can also be expressed in quadratic forms:

$$(\mu_1 - \mu_2)^T (\Sigma_1 + \Sigma_2)^{-1} (\mu_1 - \mu_2) \quad (59)$$

$$= \begin{bmatrix} M_4^A \mathbf{y} - \mu_4^A \\ M_4^B \mathbf{z} - \mu_4^B \end{bmatrix}^T \begin{bmatrix} +\Sigma_4^{-1} & -\Sigma_4^{-1} \\ -\Sigma_4^{-1} & +\Sigma_4^{-1} \end{bmatrix} \begin{bmatrix} M_4^A \mathbf{y} - \mu_4^A \\ M_4^B \mathbf{z} - \mu_4^B \end{bmatrix}, \quad (60)$$

where we denote $\Sigma_4^{-1} = (\Sigma_1 + \Sigma_2)^{-1}$.

Next, we substitute Eq. (54) and Eq. (60) into Eq. (52), focusing on the exponent term, as the remaining terms are constants. The exponent terms are summed together and can actually be combined into another quadratic form, as:

$$\begin{bmatrix} M_4^A \mathbf{y} - \mu_4^A \\ M_4^B \mathbf{z} - \mu_4^B \end{bmatrix}^T \begin{bmatrix} +\Sigma_4^{-1} & -\Sigma_4^{-1} \\ -\Sigma_4^{-1} & +\Sigma_4^{-1} \end{bmatrix} \begin{bmatrix} M_4^A \mathbf{y} - \mu_4^A \\ M_4^B \mathbf{z} - \mu_4^B \end{bmatrix} \quad (61)$$

$$+ \begin{bmatrix} \mathbf{y} - \mu_y^A \\ \mathbf{z} - \mu_z^B \end{bmatrix}^T \begin{bmatrix} \Sigma_{yy}^{A-1} & 0 \\ 0 & \Sigma_{zz}^{B-1} \end{bmatrix} \begin{bmatrix} \mathbf{y} - \mu_y^A \\ \mathbf{z} - \mu_z^B \end{bmatrix} \\ = \begin{bmatrix} \mathbf{y} - \mu_5^1 \\ \mathbf{z} - \mu_5^2 \end{bmatrix}^T \begin{bmatrix} \Sigma_5^{11} & \Sigma_5^{12} \\ \Sigma_5^{21} & \Sigma_5^{22} \end{bmatrix} \begin{bmatrix} \mathbf{y} - \mu_5^1 \\ \mathbf{z} - \mu_5^2 \end{bmatrix} \quad (62)$$

It is straightforward to solve the covariance matrix by expanding the quadratic forms. Solving quadratic terms will give us:

$$\Sigma_5^{11} = M_4^{A\top} \Sigma_4^{-1} M_4^A + \Sigma_{yy}^{A-1} \quad (63)$$

$$\Sigma_5^{22} = M_4^{B\top} \Sigma_4^{-1} M_4^B + \Sigma_{zz}^{B-1} \quad (64)$$

$$\Sigma_5^{12} = -M_4^{A\top} \Sigma_4^{-1} M_4^B \quad (65)$$

$$\Sigma_5^{21} = -M_4^{B\top} \Sigma_4^{-1} M_4^A \quad (66)$$

Meanwhile, aligning linear terms implies another linear equation:

$$\begin{bmatrix} \Sigma_5^{11} & \Sigma_5^{12} \\ \Sigma_5^{21} & \Sigma_5^{22} \end{bmatrix} \begin{bmatrix} \mu_5^1 \\ \mu_5^2 \end{bmatrix} = \begin{bmatrix} \Phi^1 \\ \Phi^2 \end{bmatrix} \quad (67)$$

where:

$$\Phi^1 = \Sigma_{yy}^{A-1} \mu_3^A + M_4^{A\top} \Sigma_4^{-1} (\mu_4^A - \mu_4^B) \quad (68)$$

$$\Phi^2 = \Sigma_{zz}^{B-1} \mu_3^B + M_4^{B\top} \Sigma_4^{-1} (\mu_4^B - \mu_4^A) \quad (69)$$

From which we can eventually solve μ_5^1 and μ_5^2 :

$$\mu_5^1 = (\Sigma_5^{11} - \Sigma_5^{12} \Sigma_5^{22-1} \Sigma_5^{21})^{-1} (\Phi^1 - \Sigma_5^{12} \Sigma_5^{22-1} \Phi^2) \quad (70)$$

$$\mu_5^2 = (\Sigma_5^{22} - \Sigma_5^{21} \Sigma_5^{11-1} \Sigma_5^{12})^{-1} (\Phi^2 - \Sigma_5^{21} \Sigma_5^{11-1} \Phi^1) = \Sigma_5^{22-1} (\Phi^2 - \Sigma_5^{21} \mu_5^1) \quad (71)$$

Notice we introduce inversion of two matrices, but since they are *Schur complement* of the covariance matrix, they are also guaranteed to be invertible as well. As a consequence, we arrive:

$$f\left(\begin{bmatrix} y \\ z \end{bmatrix}\right) = C \cdot \mathcal{N}\left(\begin{bmatrix} \mu_5^1 \\ \mu_5^2 \end{bmatrix}, \begin{bmatrix} \Sigma_5^{11} & \Sigma_5^{12} \\ \Sigma_5^{21} & \Sigma_5^{22} \end{bmatrix}^{-1}\right), \quad (72)$$

where the constant C can be easily derived to be

$$C = \frac{(2\pi)^{-0.5} \cdot |\Sigma_1 + \Sigma_2|^{-0.5}}{\sqrt{\det(\Sigma_{yy}^a) \cdot \det(\Sigma_{zz}^b) \cdot \det\left(\begin{bmatrix} \Sigma_5^{11} & \Sigma_5^{12} \\ \Sigma_5^{21} & \Sigma_5^{22} \end{bmatrix}\right)}}. \quad (73)$$

□

4 DETAILS IN PRECOMPUTED POISSON KERNEL

4.1 Computing reference Poisson kernel

In this section, we describe how the reference Poisson kernel is computed using Walk on Spheres (WoS). As noted in the main paper, we precompute the 3D Poisson kernel on a 512^3 grid. Each slice of this grid corresponds to the solution of a Dirac delta source placed on the boundary. Since the boundary domain z is discretized, we approximate the Dirac delta with a small piecewise constant source rather than a real delta function.

However, solving each slice independently for 512 times using WoS is highly inefficient, as WoS performs poorly with narrow sources. While Reverse WoS [Qi et al. 2022] handles such cases more effectively, it introduces additional bias into the solution. Therefore, we compute precomputed Poisson kernel in the following way:

- (1) We initialize a 512^3 grid with all entries set to zero.
- (2) For each pixel (x, y) , we launch multiple random walks. Whenever a walk reaches the boundary at point z , we use atomic addition to accumulate the walk's throughput into the grid element (x, y, z) .

In this way, each walk contributes to the entire grid as long as it reaches the boundary. In contrast, solving each slice independently discards any walk that lands on a boundary point with zero boundary value, resulting in inefficient use of samples. In practice, our approach produces a relatively clean Poisson kernel in about 5 minutes. This reference grid is then used to train our Gaussian models as well as all baseline methods for comparison.

5 DETAILS IN PRECOMPUTED RADIANCE TRANSFER

5.1 Obtaining slices of 4D function

In this section, we describe how we precompute the transfer functions for PRT. For each mesh vertex, we render the transfer function at a resolution of 256×256 . This already provides sufficient information to train classical PRT methods by projecting the per-vertex data onto bases such as spherical harmonics (SH), Haar wavelets,

and others. However, our method requires a different type of slice: one that effectively represents the lightmap resulting from a specific directional light. For fair comparison, we obtain these slices from per-vertex data.

Specifically, to obtain a slice $N_K(x, y, l, s_i, t_i)$, where (s_i, t_i) denotes the light direction, we discretize the Dirac delta source as a single pixel in the octahedral environment map at a resolution of 256×256 . The irradiance at each vertex can then be approximated by summing this environment map with the corresponding per-vertex transfer function. We then rasterize the mesh using texture coordinates as vertex positions and the computed irradiance as vertex colors to generate the corresponding lightmap. Note that this lightmap contains no additional information beyond the per-vertex data; thus, our method is trained with the same amount of information as the baseline approaches.

5.2 Setup of comparisons

Buddha. The BUDDHA mesh contains 20,769 vertices, for comparison:

- **Gaussian linear integral operator:** Our method uses 10,618 four-dimensional Gaussians. Each 4D Gaussian needs 17 floats (4 mean + 10 cov + 3 weight) storage.
- **Spherical Harmonics (SH):** Uses basis functions up to order 1, resulting in $4 \times 3 = 12$ coefficients per vertex (accounting for RGB channels).
- **Nonlinear Haar Wavelet:** Retains the top-3 coefficients per vertex, where each coefficient includes one index and three values (one per RGB channel).
- **Moving Basis Decomposition:** Compresses up to order-17 SH (yielding $324 \times 3 = 972$ coefficients), using 4 basis for compression. The basis functions themselves are defined on a 3^3 spatial grid, while the coefficients are stored per vertex.

The memory usage across all methods is summarized in the following table:

Ours (4DG)	SH (order-1)	Nonlinear Haar	MBD (order-17 SH)
180,506 ($\times 1$)	249,228 ($\times 1.38$)	249,228 ($\times 1.38$)	188,052 ($\times 1.04$)

Shark. The SHARK mesh contains 77,344 vertices, for comparison:

- **Gaussian linear integral operator:** our method uses 36,206 four-dimensional Gaussians.
- **Spherical Harmonics (SH):** Uses basis functions up to order 1.
- **Nonlinear Haar Wavelet:** Retains the top-3 coefficients per vertex.
- **2D Gaussian:** one lobe per vertex.
- **Moving Basis Decomposition:** Compresses up to order-10 SH (yielding $121 \times 3 = 363$ coefficients), with 8 basis functions per vertex. The basis functions are defined on a 3^3 spatial grid, while the coefficients are stored per vertex. Notice that, although SHARK is a glossy scene and intuitively might benefit from higher-order SH with fewer basis functions, in practice we observe that the signal varies too rapidly. Using too few basis functions—even with high SH order—results in poor reconstruction quality. Therefore, we end up using lower SH order with a larger number of basis functions for better result quality.

The memory usage across all methods is summarized in the following table:

Ours (4DG)	SH (order-1)	Nonlinear Haar (top-3)	2DG (1 lobe)	MBD (order-17 SH)
615,502 ($\times 1$)	928,128 ($\times 1.51$)	928,128 ($\times 1.51$)	618,752 ($\times 1.01$)	697,160 ($\times 1.13$)

Additionally, we allocate the number of Gaussians proportionally to the number of vertices in each patch. As a result, our method does not gain an advantage by using fewer Gaussians on back-facing patches and more on front-facing ones, even though the SHARK scene assumes a fixed camera view. This pushes for a fair comparison.

Leaves. For the LEAVES scene, we only learn the transfer kernel of the underlying rectangular plane. Although the geometry and parameterization appear simple, the presence of complex shadow patterns that vary significantly with lighting direction makes the fitting task very challenging.

One difference from the previous examples is that, in this case, we precompute the transfer functions on a 256×256 grid over the plane, and all methods primarily fit this grid. However, for all baseline methods (except ours), we later interpolate the resulting coefficients onto a coarser 100×100 grid. While this may slightly deviate from the principle of training all methods on the same amount of information, the adjustment is motivated by our observation that using the 256×256 grid exhibits low accuracy in the directional (input) domain, making the higher resolution in the output domain less meaningful. To improve the overall quality of the baselines, we reduce the output grid resolution while allocating more coefficients per vertex. Nonetheless, as demonstrated in Figure 16 and Figure 18 of the main paper, our method achieves superior reconstruction quality in both the output and directional domains.

Another difference is that, although the entire plane is conceptually a single patch, we manually divide it into $4 \times 4 = 16$ patches and train each patch independently. To prevent discontinuities at patch boundaries, we subsequently merge all Gaussians into a single model and do additional fine-tuning on the aggregated result.

With 10,000 vertices, for comparison:

- **Gaussian linear integral operator:** our method uses 72,649 four-dimensional Gaussians.
- **Spherical Harmonics (SH):** Uses basis functions up to order 6, resulting in $49 \times 3 = 147$ floats per vertex.
- **Nonlinear Haar Wavelet:** Retains the top-32 coefficients per vertex, thus totally $32 \times 4 = 128$ floats.
- **Moving Basis Decomposition:** Compresses up to order-12 SH (yielding $169 \times 3 = 507$ coefficients), with 80 basis functions per vertex. The basis functions are defined on a 4×4 coarser plane grid, while the coefficients are stored per vertex on the 100×100 grid. We use significantly more basis numbers given the fact the signals varies very quickly across vertices.

The memory usage across all methods is summarized in the following table:

Ours (4DG)	SH (order-6)	Nonlinear Haar (top-32)	MBD (order-17 SH)
1,234,033 ($\times 1$)	1,470,000 ($\times 1.19$)	1,280,000 ($\times 1.03$)	1,448,960 ($\times 1.17$)

5.3 Gaussian clustering.

In this section, we describe our method for clustering existing Gaussians into a smaller set to achieve a more compact representation. We largely follows Jakob et al. [2011], Algorithm 3. Two minor modifications are:

- Since our model includes RGB weights, we compute distances using the average over the RGB channels. When merging Gaussians, we update the color by averaging the RGB weights of the original Gaussians.
- Jakob et al. [2011] construct an entire pyramid of Gaussians, where each Gaussian is merged only once per layer. In contrast, our goal is to reduce the total number of Gaussians to a specified target N . Therefore, each newly merged Gaussian is immediately reinserted into the pool and remains eligible for further merging, as long as it is sufficiently close to another Gaussian.

5.4 Additional runtime timings.

We report the runtime performance of our method on a laptop equipped with an RTX 3070 GPU in Table 1.

Table 1. RUNTIME PERFORMANCE. We measure the runtime frametime (ms) in our actual renderer running on an RTX 3070 Laptop GPU, across various scenes, and numbers of input (environment) Gaussians.

Resolution	Scene	# input 2DGS	frametime (ms)
512	Buddha	3	5.68
		10	18.52
		20	30.30
		40	41.67
	Shark	3	6.25
		10	20.41
		20	33.33
		40	50.00
	Leaves	3	270.27
		10	2,173.9
		20	-
		40	-

5.5 Fitting environment maps with 2D Gaussians.

Fitting 2D Gaussians for environment maps is orthogonal to our main contribution. For this example, we use the differentiable rasterizer combined with iterative clustering to optimize the Gaussian parameters. This fitting process, along with coefficient computation for baselines, is performed offline as a precomputation step.

Since we aim to compare operator precision under equal memory constraints, we seek to minimize error introduced by the environment map approximation. The number of Gaussians is manually selected based on map complexity, guided by two criteria: 1) Input approximation error should not be the main source of output quality degradation, 2) The total storage for approximating the environment map should remain below that of MBD, which uses the most coefficients across baselines. We use 3, 20, or 40 Gaussians for the environment maps in Table 4. For the more complex map in the first row of Fig. 15, we use 100 Gaussians.

6 DETAILS IN SPECTRAL RENDERING

6.1 Implementation of Gaussian spectrum and fluorescence

Obtaining Gaussian approximations. For re-radiation matrices, we directly adopt the Gaussian mixtures produced by Hua et al. [2023], who proposed approximating re-radiation matrices using Gaussian mixtures. For general spectra, we extract dense spectral data from PBRT and fit 1D Gaussian mixture models (GMMs) using `scipy`. All approximation is done offline before any rendering starts.

Backward form. To implement the backward form of our method, we use a vector to record pointers to the materials encountered along the entire path, until a light source is reached. Since we employ Next Event Estimation (NEE), we connect to the light at each path vertex.

When connecting to or hitting the light, we retrieve the 1D Gaussian spectrum of the light's emission. We then traverse the recorded material vector in reverse order, reading the corresponding operator Gaussians and applying them sequentially to the input spectrum, propagating the result toward the camera. At the camera, we analytically integrate the final spectrum with the CIE XYZ color matching functions, which are also approximated as Gaussian mixtures, to compute the final color response.

After applying each operator, a large number of output Gaussians are typically generated. In our stochastic variant, we do not retain all of these Gaussians for the next operator. Instead, we apply weighted reservoir sampling to traverse the output Gaussians and select 4 samples to serve as the input for the next stage. The sampling process uses a custom importance criterion, which is approximately equivalent to luminance. Thus the number of Gaussians would not grow exponentially with larger path lengths.

Forward form. To implement the forward form of our method, we compose the operators corresponding to all materials along the light path. In general, each operator consists of a diagonal component represented by 1D Gaussians, and an off-diagonal component represented by 2D Gaussians.

Composing two 2D Gaussians results in a new 2D Gaussian. Similarly, composing a 2D Gaussian with a 1D diagonal Gaussian also yields a new 2D Gaussian. In contrast, composing two 1D diagonal Gaussians produces a new 1D diagonal Gaussian. Therefore, to represent the composed operator, we maintain two separate vectors: one for storing all 2D Gaussians and another for all 1D Gaussians. When the path hit or connect to the lights, we just apply the recorded operator to the light Gaussian spectra.

In the stochastic variation of our method, we similarly avoid storing all composed Gaussians. Instead, we maintain a fixed set of four 2D Gaussians and four 1D Gaussians. After each composition step, we apply weighted reservoir sampling (WRS) to select four 2D and four 1D Gaussians from the generated outputs, ensuring a compact yet representative approximation.

REFERENCES

- Qingqin Hua, Vojtěch Tázlar, Alban Fichet, and Alexander Wilkie. 2023. Efficient Storage and Importance Sampling for Fluorescent Reflectance. *Computer Graphics Forum* 42, 1 (2023), 47–59. <https://doi.org/10.1111/cgf.14716>
- Wenzel Jakob, Christian Regg, and Wojciech Jarosz. 2011. Progressive Expectation-Maximization for Hierarchical Volumetric Photon Mapping. *Computer Graphics Forum (Proceedings of Eurographics Symposium on Rendering)* 3, 4 (July 2011), 1287–1297. <https://doi.org/10.1111/j.1467-8659.2011.01988.x>
- Yang Qi, Dario Seyb, Benedikt Bitterli, and Wojciech Jarosz. 2022. A bidirectional formulation for Walk on Spheres. *Computer Graphics Forum (Proceedings of EGSR)* 41, 4 (July 2022). <https://doi.org/10/jgzs>